

Seguridad Web I

Inyección, XSS y Control de Acceso

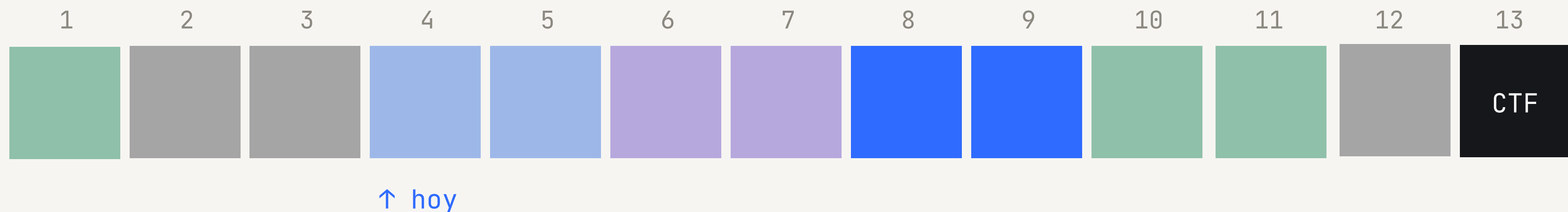
Maico Molina Aoki

FaMAF · UNC · 2026 · Semana 2

Hoy

- 1 Repaso: cómo funciona una app web
- 2 Inyección: SQLi & Command Injection
- 3 XSS & CSRF
- 4 Control de Acceso Roto ← rompé la lógica, no el candado
- 5 Herramientas: Burp, sqlmap, DevTools
- 6 Práctica en CTFd (60 min)

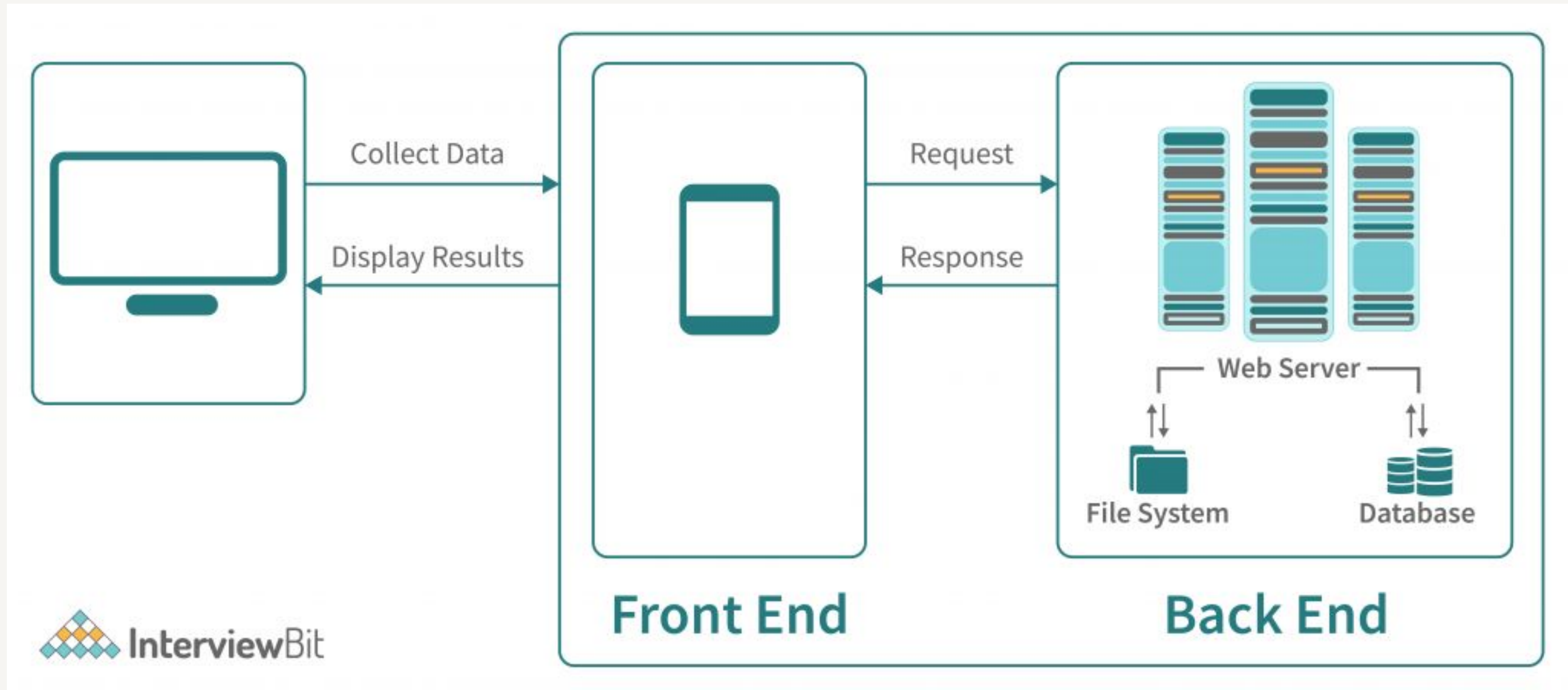
13 Semanas · 19 ago → 11 nov



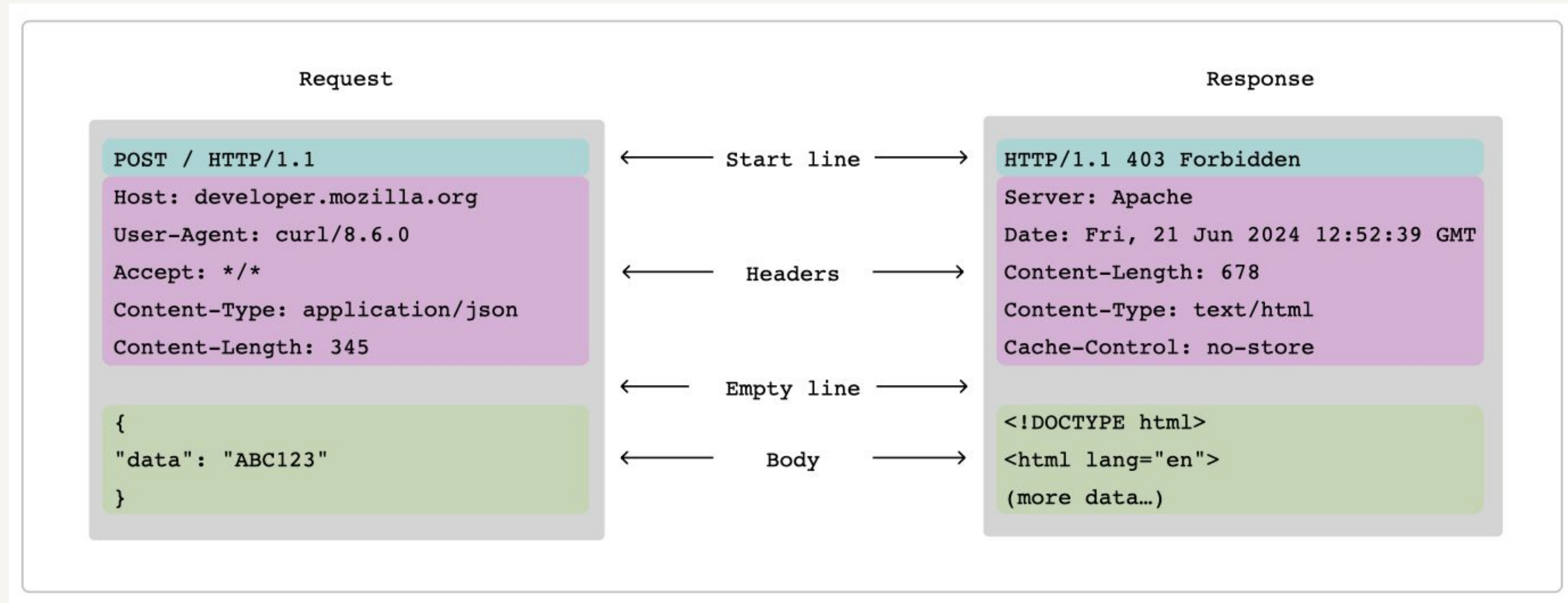
Web Reversing Pwn Pentest Comodin CTF final

Cómo funciona una app web

Front - Back Model



HTTP Messages



**Reconocimiento
antes de explotar**

Antes de inyectar nada: mirá primero

- Ver el código fuente (Ctrl+U) y los comentarios
- Inspeccionar Network tab: requests, headers, cookies
- Fuzzear rutas y parámetros (ffuf, gobuster)
- Leer robots.txt, sitemap.xml, JS bundles

Inyección

SQLi: consulta vulnerable vs. segura

SEGURO

Prepared statements / queries parametrizadas

El input nunca se interpreta como código SQL

Funciona igual con ORMs bien configurados

→ el ' del atacante es sólo un carácter más

VULNERABLE

```
"SELECT * FROM users WHERE  
user=" + input + ""
```

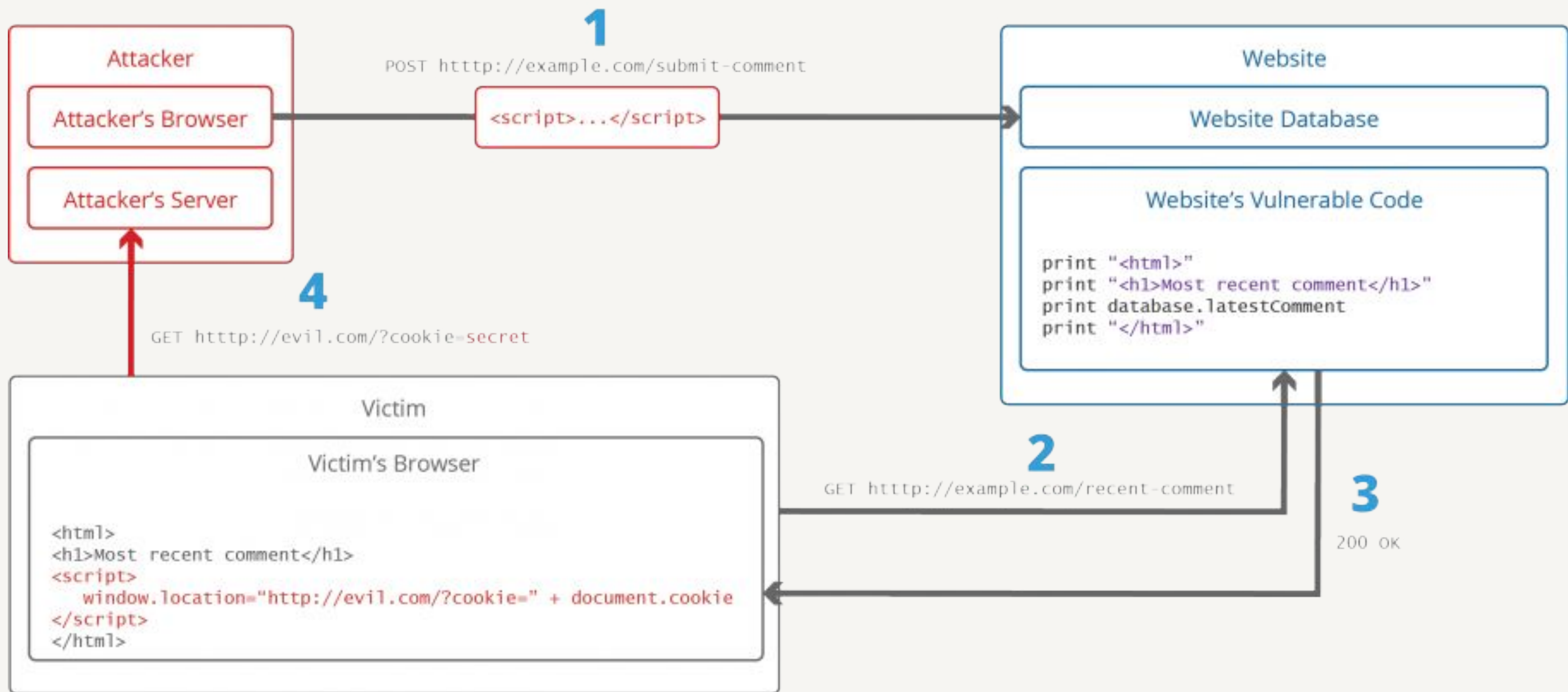
El input se concatena directo en la query

Un ' cierra el string e inyecta lógica SQL → bypass de login, dump de tablas

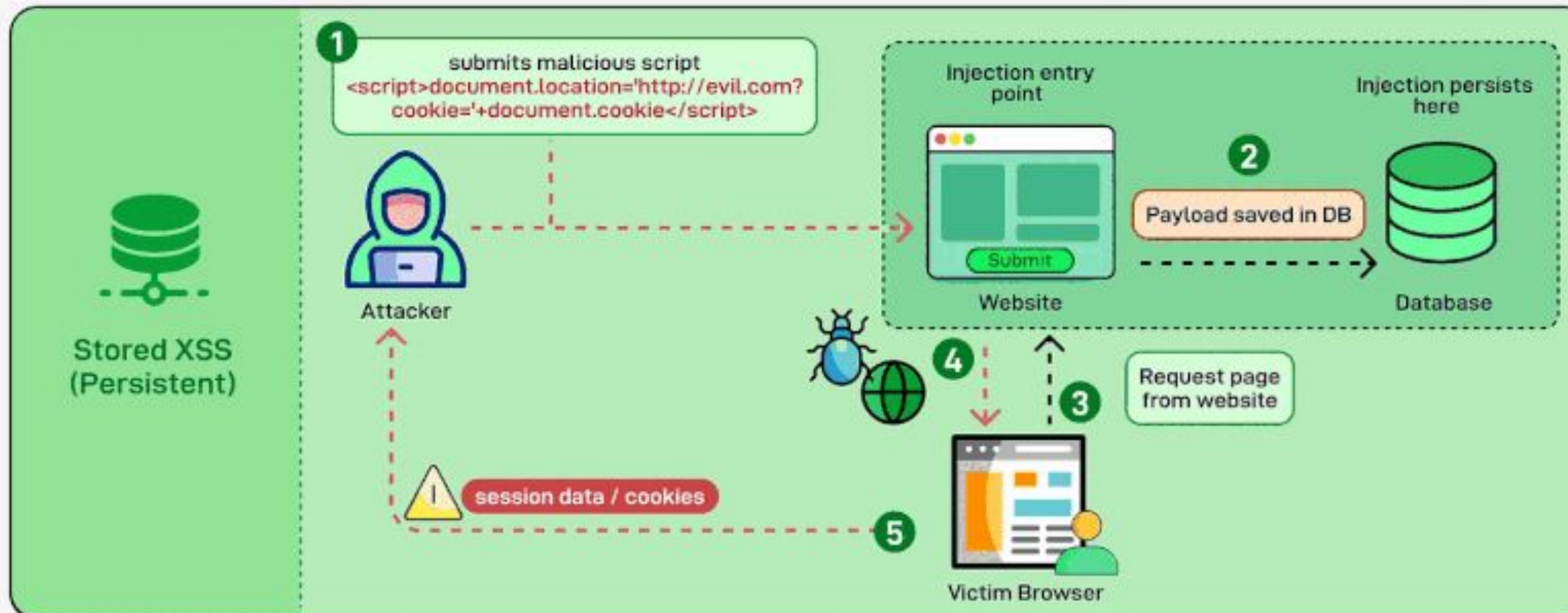
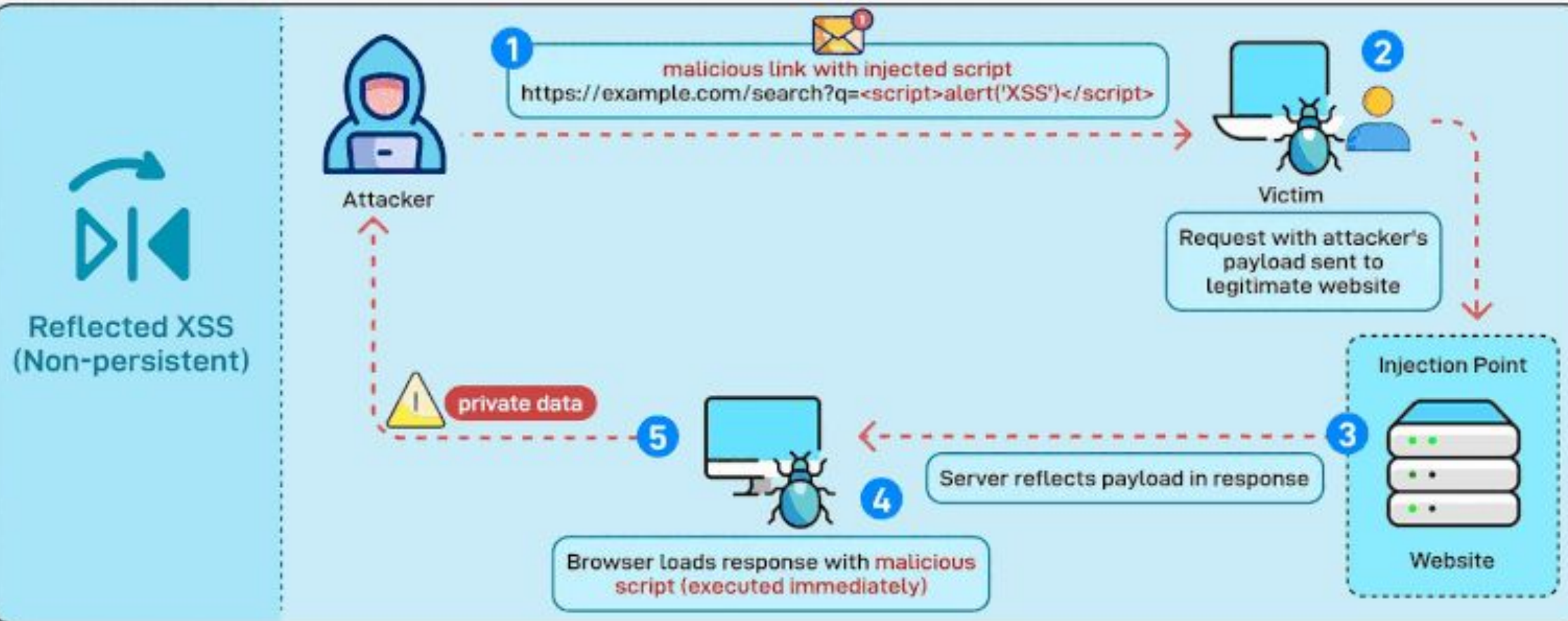
Command Injection: mismo patrón, otro intérprete

- ping, whoami, cat, curl — cualquier función que arme
- `import __import__ ` → comando extra para el`
- `system('process.r`
`un(cmd,`
- **Mitigación:** listas de **agrandamiento** **especial** **chequeo** con input externo
bug

XSS & CSRF



Cross-Site Scripting (XSS) Attacks



Tres sabores de XSS

REFLEJADO

Viaja en la URL o un parámetro

Se ejecuta al abrir un link armado

No persiste en el servidor

ALMACENADO

Se guarda en la BD (comentario, perfil, etc.)

Se ejecuta para todos los que ven esa página

El más peligroso de los tres

DOM-BASED

Nunca toca el servidor

El JS del cliente inserta input sin sanitizar

innerHTML, document.write, eval son sospechosos

↑ CSRF es un problema relacionado: el navegador manda cookies solo

Control de Acceso Roto

IDOR: vulnerable vs. seguro

SEGURO

El server valida: ¿este user_id es el dueño del recurso?
IDs impredecibles (UUID) ayuda, pero no reemplaza la validación
Autorización se chequea en CADA request
→ autenticado ≠ autorizado

VULNERABLE

GET /invoice?id=1042 → cambio a id=1043 y veo la factura de otro

El server confía en que el ID es válido porque el usuario está logueado

Nunca se compara el dueño del recurso contra el usuario actual

Herramientas de esta clase

- ▶ Burp Suite / OWASP ZAP — interceptar y modificar requests
- ▶ DevTools del navegador — Network, Storage, Console
- ▶ sqlmap — automatizar la explotación de SQLi
- ▶ curl / httpie — armar requests a mano

A practicar

Con el mail que usaron en el form!



<https://208-113-208-13.sslip.io/register>

Ahora ustedes

60:00

Web I –
Challenges

Recon Warmup

SQLi 101

Command Injection

XSS Playground

IDOR: Invoice Hub

IDOR: NorthGate
Bank

Por qué empezamos por acá

- Inyección y XSS siguen siendo las más comunes en apps reales
- Control de acceso roto es la #1 del OWASP Top 10 (A01)
- Son fáciles de demostrar con impacto real
- Todo lo que viene (auth, SSRF, etc.) se apoya en esta base

Cómo nos contactan

<https://discord.gg/EKUtebwMe>

← preferido, contestamos más rápido

`gastonaznarez@mi.unc.edu.ar`

`maico.molina@mi.unc.edu.ar`



Nos vemos en la Semana 3